BÁO CÁO BÀI THỰC HÀNH NÂNG CAO: LẬP TRÌNH MẠNG VỚI APACHE KAFKA

Học phần Lập trình mạng (Lớp 2627I_INT3304_3)

· Giảng viên: ThS. Kiều Hữu Thu

· Sinh viên: Nguyễn Khắc Huy · MSSV: 24022802

1. Giới thiệu và mục tiêu

Apache Kafka là nền tảng phát và lưu trữ luồng sự kiện (event streaming) theo mô hình Pub/Sub bất đồng bộ. Khác với mô hình client/server đồng bộ, bên gửi (Producer) và bên nhận (Consumer) không nối trực tiếp với nhau mà cùng làm việc với Kafka broker, nơi lưu event và phân phối cho các bên đọc. Bài thực hành dựng một cluster nhỏ gồm 02 broker logic trên một máy, tạo topic nhiều partition có replication, viết Java Producer/Consumer và quan sát các đặc điểm: giao tiếp bất đồng bộ, Pub/Sub giữa các Consumer Group và chia tải theo partition.

2. Kiến trúc và môi trường

```
Producer (Java)  --publish-->  Kafka cluster (Broker 1 :9092, Broker 2 :9094)  --poll-->  Consumer (Java)
                                        Topic: network-events (2 partition, replication 2)
```

Thành phần	Giá trị
Máy ảo	VMware, Ubuntu Server [phiên bản], RAM [4 GB], CPU [2 nhân]
Java	OpenJDK 17
Maven	3.x
Apache Kafka	4.3.1 (chế độ KRaft, không dùng Zookeeper)
Công cụ	tmux (chạy nhiều cửa sổ), SSH từ PowerShell

Node	Broker port	Controller port	node.id	Data directory
Broker 1	9092	9093	1	~/kafka-data/node1
Broker 2	9094	9095	2	~/kafka-data/node2

Đây là mô hình hai node logic trên cùng một host, phù hợp để học cấu trúc cluster và lập trình client. Đây không phải hệ thống phân tán vật lý: nếu máy ảo tắt thì cả hai broker cùng dừng.

3. Cấu hình và khởi chạy cluster

Hai file cấu hình `lab-node1.properties` và `lab-node2.properties` đặt trong thư mục `config` của Kafka. Các khóa chính: `process.roles=broker,controller`, `listeners` và `advertised.listeners` theo bảng cổng ở trên, `controller.quorum.voters=1@localhost:9093,2@localhost:9095`, `log.dirs` riêng cho mỗi node.

So với đề bài, em thêm 3 dòng vào cả hai file:

```
offsets.topic.replication.factor=2
transaction.state.log.replication.factor=2
transaction.state.log.min.isr=1
```

Lý do: mặc định Kafka tạo các topic nội bộ (như `__consumer_offsets`) với 3 bản sao, trong khi cluster chỉ có 2 broker, nên Consumer có thể bị treo hoặc lỗi khi tham gia group.

Các bước khởi chạy: sinh Cluster ID bằng `kafka-storage.sh random-uuid`, format cả hai node bằng **cùng một Cluster ID** (`kafka-storage.sh format -t <ID> -c <config>`), sau đó chạy hai broker bằng `kafka-server-start.sh` trong tmux. Với quorum 2 controller, phải chạy đủ cả hai broker thì cluster mới hoạt động. Đây là cấu hình chỉ dùng cho lab, không phải kiểm thử failover.

[Chèn ảnh: config.png - nội dung hai file cấu hình]

[Chèn ảnh: cluster.png - hai broker đang chạy, kết quả jps]

4. Tạo và kiểm tra topic

```
bin/kafka-topics.sh --create --topic network-events --partitions 2 --replication-
factor 2 --bootstrap-server localhost:9092
bin/kafka-topics.sh --describe --topic network-events --bootstrap-server
localhost:9092
```

Partition	Leader	Replicas	ISR
0	1	1, 2	1, 2
1	2	2, 1	2, 1

[Ảnh: topic.png - kết quả lệnh describe]

Topic có 2 partition, mỗi partition có một leader và một bản sao ở broker còn lại. ISR gồm cả hai broker nghĩa là bản sao đang đồng bộ.

5. Thử Producer/Consumer bằng dòng lệnh

Dùng `kafka-console-producer.sh` gửi bốn dòng `Hello Kafka`, `Student-01`, `network-programming`, `async-event` và `kafka-console-consumer.sh --from-beginning` nhận lại đủ bốn dòng. Kết quả chứng minh cluster hoạt động trước khi viết code.

[ảnh: console-producer.png]

[ảnh: console-consumer.png]

6. Lập trình Java Producer/Consumer

Project Maven `kafka-network-lab` dùng thư viện `kafka-clients` 4.3.1 và `slf4j-simple`. Cấu trúc:

```
kafka-network-lab/
├── pom.xml
└── src/main/java/
    ├── EventProducer.java
    ├── EventConsumer.java
    ├── EventProducer20.java
    └── EventProducerBulk.java
```

- `EventProducer`: đọc từng dòng từ bàn phím, gửi vào topic `network-events`, callback in `partition` và `offset` của bản ghi đã gửi.
- `EventConsumer`: subscribe topic, `poll` liên tục và in nội dung, partition, offset. Nhận `group.id` qua tham số dòng lệnh (mặc định `network-lab-group`) để dùng cho các thí nghiệm Pub/Sub và chia tải. Đặt `auto.offset.reset=earliest`.
- `EventProducer20`: gửi 20 event kèm key để cả hai partition đều có dữ liệu (thí nghiệm 4).
- `EventProducerBulk`: gửi 10.000 event và đo thời gian (thí nghiệm 10.000 event).

7. Các thí nghiệm

Thí nghiệm 1: Producer → Kafka → Consumer

Chạy EventConsumer rồi EventProducer, gửi `event-001`, `event-003`, `event-002` (đúng thứ tự thực tế đã gõ).

Event	Partition	Offset
event-001	0	4
event-003	0	5
event-002	0	6

[ảnh: producer.png]

[ảnh: consumer.png]

Nhận xét: cả ba event cùng vào partition 0 vì Producer gửi không có key, Kafka không chia đều giữa các partition. Offset bắt đầu từ 4 vì partition 0 đã có 4 bản ghi (offset 0 đến 3) từ bước thử console. Offset là số thứ tự tăng dần trong từng partition.

Thí nghiệm 2: Giao tiếp bất đồng bộ

Dừng Consumer nhưng giữ nguyên hai broker; Producer gửi `offline-event-001..003` và vẫn nhận phản hồi `Sent: partition=0` với offset 7, 8, 9, không báo lỗi. Khởi động lại Consumer cùng `group.id`, Consumer nhận đúng ba event này mà không đọc lại các event đã xử lý trước đó.

[ảnh: tn2-consumer-stop.png]

[ảnh: tn2-producer-offline.png]

[ảnh: tn2-consumer-restart.png]

Giải thích: Producer chỉ ghi vào broker và nhận xác nhận từ broker, không cần Consumer đang chạy. Event được lưu trên đĩa của broker; Consumer đọc theo nhịp riêng, tiếp tục từ offset đã commit của group.

Thí nghiệm 3: Pub/Sub với hai Consumer Group

Chạy Consumer A với `group.id=network-lab-group` và Consumer B với `group.id=monitor-group`, sau đó gửi `PUBSUB001..003` (partition 0, offset 10, 11, 12). Cả hai group đều nhận đủ ba event. Consumer B là group mới nên còn đọc lại toàn bộ event cũ trong topic (do `earliest`).

[ảnh: tn3-producer.png]

[ảnh: tn3-consumer-A.png]

[ảnh: tn3-consumer-B.png]

Giải thích: mỗi Consumer Group giữ offset riêng trên topic, nên cùng một event được giao cho mọi group. Các group khác nhau hoạt động như các subscriber độc lập.

Thí nghiệm 4: Chia tải trong cùng một Consumer Group

Chạy hai Consumer cùng `group.id=network-lab-group`. Vì Producer không key đưa mọi event vào partition 0 (quan sát ở thí nghiệm 1), em dùng `EventProducer20` gửi 20 event có key `key-1..key-20` để cả hai partition đều có dữ liệu. Producer báo: partition 0 nhận 9 event, partition 1 nhận 11 event.

Consumer	Partition được giao	Số event nhận
Consumer 1	1	11 (load-003, 004, 007, 008, 009, 011, 013, 015, 016, 018, 020)
Consumer 2	0	9 (load-001, 002, 005, 006, 010, 012, 014, 017, 019)
Tổng		20

[ảnh: tn4-producer.png]

[ảnh: tn4-consumer1.png]

[ảnh: tn4-consumer2.png]

[ảnh: tn4-describe-group.png - kết quả kafka-consumer-groups.sh --describe, mỗi partition có CONSUMER-ID khác nhau]

Giải thích: topic có 2 partition và group có 2 Consumer nên Kafka giao mỗi partition cho đúng một Consumer; mỗi event chỉ được một Consumer của group xử lý. Nếu thêm Consumer thứ 3 vào cùng group, nó sẽ không có partition để nhận, vì số Consumer hữu ích tối đa bằng số partition.

8. Thí nghiệm lưu lượng 10.000 event

`EventProducerBulk` gửi 10.000 event bằng `producer.send()` không chờ từng kết quả, sau đó `flush()` rồi tính thời gian và throughput.

Lần chạy	Elapsed (s)	Throughput (events/s)
1	0,513	19493.17738791423
2	0.442	22624.43438914027
3	0.413	24213.07506053269

[ảnh: tn5-10000-3lan.png]

Kiểm tra tính đầy đủ: sau ba lần chạy, offset cuối của partition 0 là 9011 và của partition 1 là 21022 (tổng 30.033 = 33 event trước đó + 3 × 10.000). Group `network-lab-group` có LAG tổng 30.000 (8.989 + 21.011), khớp đúng số event đã gửi, nghĩa là không mất event nào.

[ảnh: tn5-10000-code-offsets.png]

[ảnh: tn5-group-lag.png]

Nhận xét: `send()` bất đồng bộ, Producer gom nhiều bản ghi thành lô rồi mới gửi lên broker nên throughput cao hơn nhiều so với gửi từng event. Các event không key bị dồn theo lô vào từng partition nên phân bố không đều (partition 0 nhận 8.989, partition 1 nhận 21.011 trong 30.000 event). Số liệu chỉ mang tính gần đúng trên máy ảo có cả hai broker và client cùng chạy, không dùng để so sánh hiệu năng với máy khác.

9. Trả lời câu hỏi thảo luận

a) Kafka khác mô hình TCP socket Client/Server trực tiếp ở điểm nào? Với socket, client nối thẳng tới server, cả hai phải cùng online và client phải biết địa chỉ server. Với Kafka, Producer và Consumer đều làm việc với broker; broker lưu event trên đĩa nên bên gửi và bên nhận tách rời nhau về thời gian và địa chỉ.

b) Producer có cần biết địa chỉ IP của Consumer không? Vì sao? Không. Producer chỉ cần địa chỉ broker (`bootstrap.servers`) và tên topic. Ai đọc, đọc khi nào và bao nhiêu Consumer đều do phía Consumer quyết định.

c) Consumer tạm thời không chạy thì Producer có bắt buộc phải dừng theo không? Không. Ở thí nghiệm 2, Consumer bị tắt nhưng Producer vẫn gửi `offline-event-001..003` thành công; khi Consumer chạy lại thì nhận đủ vì event đã được broker lưu.

d) Topic và Partition khác nhau như thế nào? Topic là luồng dữ liệu có tên (`network-events`); Partition là một phần của topic, là một dãy event có thứ tự với offset tăng dần và được lưu, sao chép độc lập. Thứ tự event chỉ được đảm bảo trong từng partition.

e) Tại sao nhiều Partition giúp tăng khả năng xử lý song song? Mỗi partition được giao cho đúng một Consumer trong một group tại một thời điểm, nên nhiều partition cho phép nhiều Consumer đọc song song; phía ghi, các partition nằm trên nhiều broker nên cũng ghi song song. Ở thí nghiệm 4, hai Consumer cùng group chia nhau hai partition.

f) Hai Consumer cùng group.id khác gì hai Consumer có group.id khác nhau? Cùng group: chia nhau các partition, mỗi event chỉ một Consumer nhận (chia tải, thí nghiệm 4). Khác group: mỗi group giữ offset riêng và nhận toàn bộ event, như các subscriber độc lập (Pub/Sub, thí nghiệm 3).

g) Nêu 03 bài toán thực tế phù hợp với Kafka. (1) Thu thập và giám sát log tập trung từ nhiều dịch vụ: nhiều nguồn ghi liên tục, nhiều hệ thống cùng đọc. (2) Luồng giao dịch ngân hàng, thanh toán (mobile banking, core banking): lưu lượng lớn, cần lưu bền vững và xử lý bất đồng bộ. (3) Dữ liệu IoT/telemetry hoặc giám sát mạng viễn thông 4G/5G: rất nhiều thiết bị gửi sự kiện liên tục cho nhiều ứng dụng phân tích.

h) Kafka có thay thế hoàn toàn HTTP/REST hoặc TCP socket không? Không. Kafka phù hợp với luồng sự kiện bất đồng bộ, lưu lượng lớn. HTTP/REST và socket vẫn phù hợp với tương tác request/response đồng bộ cần phản hồi ngay (ví dụ gọi API đăng nhập, tra cứu). Hai loại thường bổ sung cho nhau trong cùng một hệ thống.

10. Nhận xét và khó khăn gặp phải

Nhận xét về Kafka. Kafka tách rời bên phát sinh và bên xử lý event, lưu event bền vững nên Consumer có thể đọc lại hoặc đọc muộn, và dễ mở rộng bằng partition và Consumer Group. Đổi lại, hệ thống phức tạp hơn socket/REST (cần broker, cấu hình, quản lý offset), và thứ tự event chỉ được đảm bảo trong từng partition.

Khó khăn và cách xử lý.

- Lệnh bị lặp lỗi `Node may not be available`: do broker chưa chạy hoặc đã tắt. Cần kiểm tra bằng `jps` thấy đủ hai tiến trình `Kafka` trước khi chạy Producer/Consumer.
- Với quorum 2 controller, một broker đứng một mình không mở cổng; phải chạy cả hai broker. Vì vậy mô hình này chỉ để học, không đánh giá được tính sẵn sàng cao.
- Topic nội bộ mặc định cần 3 bản sao: xử lý bằng 3 dòng `replication.factor` bổ sung ở mục 3.
- Event không key dồn vào một partition: xử lý bằng cách gửi kèm key ở thí nghiệm 4.
- Chưa thực hiện các bài mở rộng (JSON event, nhiều Producer, Monitor Consumer, chạy qua 2 máy LAN).

11. Kết luận

Bài thực hành đã dựng được cluster Kafka 2 broker KRaft trên một máy, tạo topic 2 partition với replication factor 2, viết Java Producer/Consumer và kiểm chứng ba đặc điểm chính: giao tiếp bất đồng bộ, Pub/Sub giữa các group, và chia tải theo partition trong cùng group, cùng thí nghiệm gửi 10.000 event không mất dữ liệu.